AI Customer Support Agent — Project Plan

E-Commerce Refund Processing System

Prepared by: Meghana Lopinti
Scope: Full-Stack Agentic AI Application (Vertical Slice)
Timeline: 7 Days
Deliverable: Loom Video + GitHub Repository

---

1. Executive Summary

Build a production-grade **AI Customer Support Agent** that handles e-commerce refund requests through an intelligent agent loop. The system will ingest customer refund requests, validate them against a strict refund policy via RAG and tool calling, approve standard refunds, and "hold the line" on policy violations. The deliverable includes a customer-facing chat/voice interface, an admin dashboard with real-time reasoning logs, and full MLOps observability.

Key Differentiator: This project directly maps to your existing expertise in **LangGraph multi-agent architectures**, **FastAPI microservices**, **Streamlit UIs**, **RAG pipelines**, and **MLOps observability** (MLflow, Prometheus, Grafana).

---

2. Problem Statement

E-commerce customer support teams face:

- **High volume** of repetitive refund inquiries
- **Inconsistent policy enforcement** due to human agent variability
- **Slow resolution times** leading to poor customer experience
- **No audit trail** of why certain refunds were approved or denied

Solution: An autonomous AI agent that enforces policy consistently, explains its reasoning transparently, and escalates edge cases—all while providing real-time observability to admins.

---

3. Core Use Cases

Use Case	Description	Expected Outcome
UC-1: Standard Refund	Customer requests refund within policy window, item unused, receipt available	Agent approves refund, generates confirmation, updates CRM
UC-2: Policy Violation ("Hold the Line")	Customer requests refund outside window, item used, or no receipt	Agent denies refund, cites specific policy clause, offers alternative (store credit)
UC-3: Edge Case / Ambiguity	Request is borderline (e.g., 1 day past deadline, defective item)	Agent initiates "human escalation" workflow with summarized context

Table 1 – continued

Use Case	Description	Expected Outcome
UC-4: Voice Interaction (Bonus)	Customer speaks refund request via microphone	Agent transcribes (Whisper), processes, responds via voice (ElevenLabs)

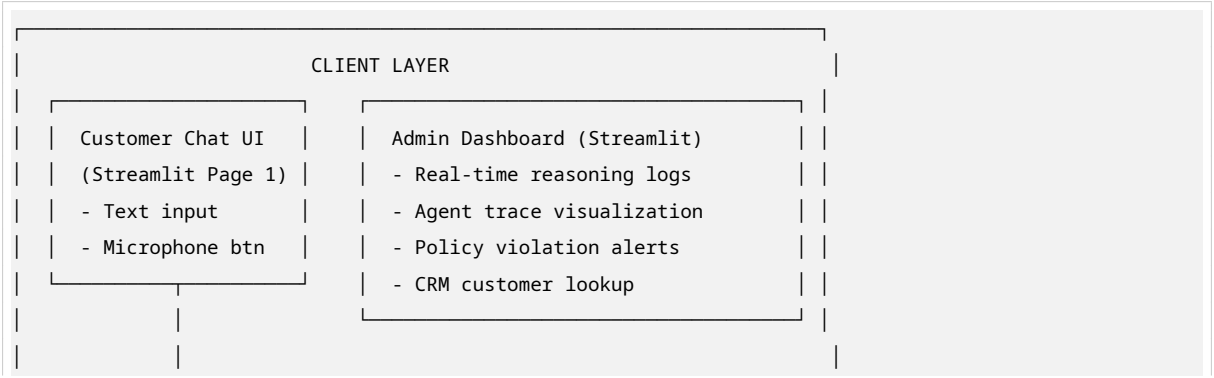
---

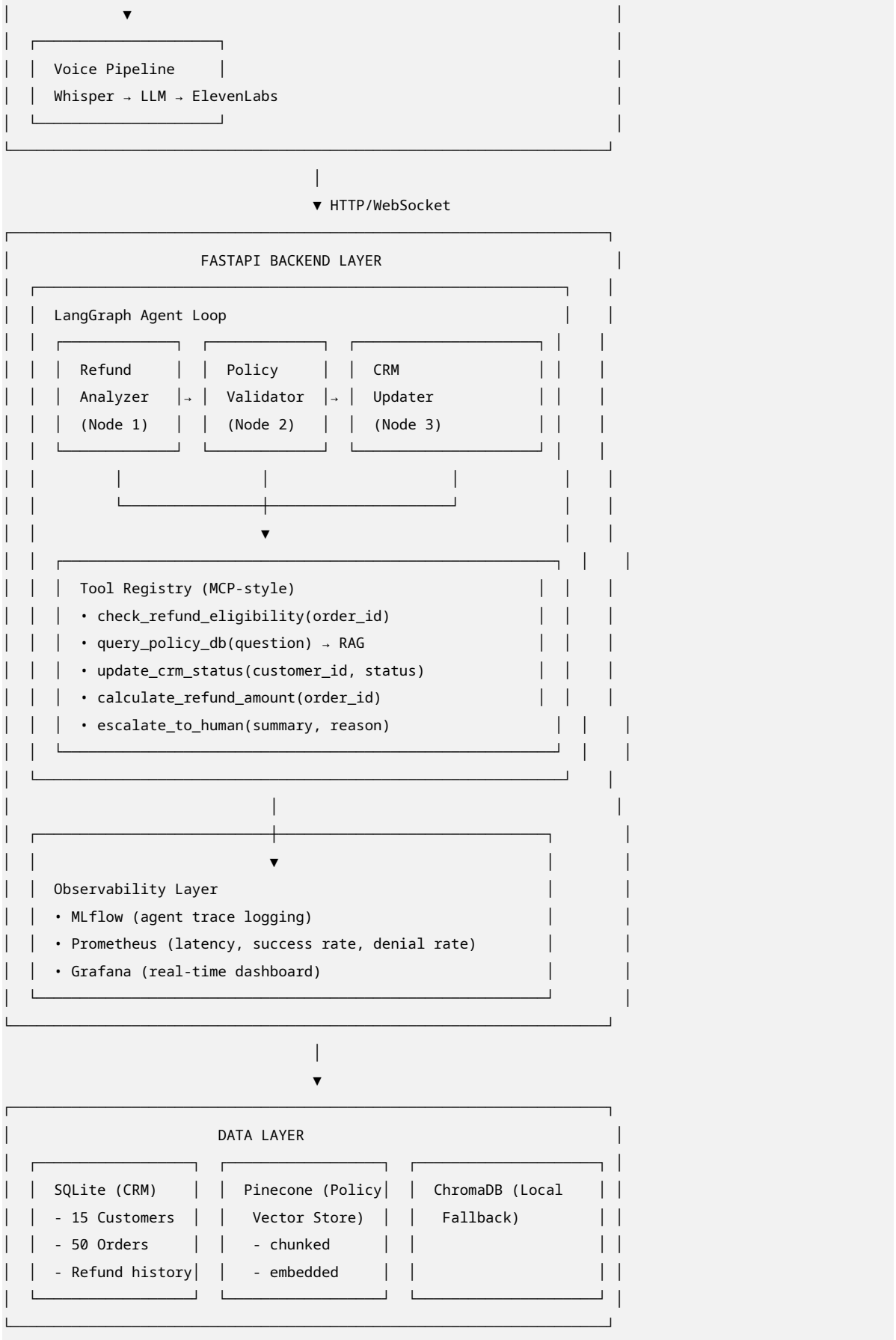
4. Tech Stack (Resume-Aligned)

Layer	Technology	Resume Mapping
LLM & Orchestration	Groq LLM (LLaMA 3.3 70B) + LangGraph	Proven in Travel Planner; low latency, cost-effective
RAG / Vector DB	Pinecone + Hugging Face Embeddings	Enterprise Policy Assistant project
Policy Ingestion	LangChain Document Loaders + Recursive Chunking	Enhanced retrieval accuracy by 35% in past project
Agent Tools	Python Functions + MCP (Model Context Protocol)	Implemented MCP in Travel Planner, cut integration code by 60%
Backend API	FastAPI (async)	Microservices architecture experience
Frontend	Streamlit (Customer Chat + Admin Dashboard)	Deployed Streamlit + FastAPI on Render previously
Voice Pipeline	OpenAI Whisper (STT) + ElevenLabs (TTS)	Whisper API experience in Multimodal RAG
Database	SQLite (Mock CRM) + ChromaDB (local fallback)	SQL expertise, vector DB experience
MLOps & Observability	MLflow (tracing), Prometheus (metrics), Grafana (dashboards)	End-to-end MLOps workflow in past project
CI/CD	GitHub Actions	Automated release cycles, accelerated by 60%
Deployment	Docker + Render	Deployed on Render with 512MB constraint optimization
Cloud (Optional)	Azure OpenAI / Azure Cognitive Services	Azure AI Engineer (AI-102) certified; primary work experience

---

5. System Architecture




▼
Voice Pipeline
Whisper → LLM → ElevenLabs
▼ HTTP/WebSocket
FASTAPI BACKEND LAYER
LangGraph Agent Loop
Refund Analyzer (Node 1)
Policy Validator (Node 2)
CRM Updater (Node 3)
Tool Registry (MCP-style)
• check_refund_eligibility(order_id)
• query_policy_db(question) → RAG
• update_crm_status(customer_id, status)
• calculate_refund_amount(order_id)
• escalate_to_human(summary, reason)
Observability Layer
• MLflow (agent trace logging)
• Prometheus (latency, success rate, denial rate)
• Grafana (real-time dashboard)
▼
DATA LAYER
SQLite (CRM)
- 15 Customers
- 50 Orders
- Refund history
Pinecone (Policy Vector Store)
- chunked
- embedded
ChromaDB (Local Fallback)

6. Data Layer Design

6.1 Mock CRM Database (SQLite)

15 Customer Profiles with:

- `customer_id`, `name`, `email`, `tier` (Bronze/Silver/Gold)
- `order_id`, `order_date`, `product_category`, `purchase_amount`
- `refund_history` (count, last refund date, fraud flag)
- `shipping_status`, `return_window_days`

Seed Data Strategy:

- 5 customers with clean refund history (standard case)
- 5 customers with edge cases (1 day past window, opened package, etc.)
- 5 customers with policy violations (past 30+ days, digital goods, etc.)

6.2 Refund Policy Document

A strict, realistic e-commerce policy (3-4 pages) covering:

- Time limits (30 days physical, 14 days digital, 7 days perishable)
- Condition requirements (unopened, original packaging, receipt required)
- Category exclusions (final sale, intimate apparel, gift cards)
- Tier-based exceptions (Gold members get +15 days)
- Fraud prevention (max 3 refunds/year, velocity checks)

Ingestion Pipeline:

1. Load PDF/Markdown policy doc
2. LangChain RecursiveCharacterTextSplitter (chunk size 512, overlap 50)
3. Hugging Face `all-MiniLM-L6-v2` embeddings
4. Upsert to Pinecone index with metadata filters (category, time_limit)

---

7. Backend Design: Agent Loop (LangGraph)

7.1 Graph State Schema

```
# Conceptual state (not actual code)
State = {
    "customer_id": str,
    "order_id": str,
    "user_query": str,
    "transcribed_voice_text": str | None,
    "retrieved_policy_clauses": list[Document],
    "crm_lookup_result": dict,
    "eligibility_check": dict,  # {eligible: bool, reason: str, confidence: float}
    "agent_decision": str,        # "APPROVE" | "DENY" | "ESCALATE"
```

```
    "response_to_customer": str,
    "reasoning_log": list[str],   # For admin dashboard
    "escalation_summary": str | None,
    "trace_id": str               # For MLflow correlation
}
```

7.2 Agent Nodes

Node	Function	Tools Used
intake	Parse request, identify customer/ order, transcribe voice if needed	`parse_natural_language`, `whisper_transcribe`
crm_lookup	Fetch customer profile, order details, refund history	`get_customer_profile`, `get_order_details`
policy_retrieval	RAG search for relevant policy clauses	`query_policy_vectorstore`
eligibility_evaluator	Apply business rules, calculate refund amount	`check_refund_eligibility`, `calculate_refund_amount`
decision	LLM reasoning to approve/deny/ escalate	None (pure LLM reasoning with structured output)
action	Update CRM, send response, trigger escalation	`update_crm_status`, `escalate_to_human`, `generate_voice_response`
logging	Emit trace to MLflow, Prometheus metrics	`log_trace`, `emit_metrics`

7.3 Conditional Edges

- `eligibility_check.confidence > 0.9` → decision node
- `eligibility_check.confidence < 0.9` → escalate node
- `fraud_flag == True` → immediate escalate
- `customer_tier == "Gold" AND borderline_case` → decision node (lenient)

7.4 Tool Registry (MCP-Style)

Instead of hardcoding tools, define a `ToolServer` class with standardized schemas—mirroring your Travel Planner MCP architecture. This demonstrates advanced integration patterns.

---

8. Frontend Design (Streamlit)

8.1 Customer Interface (`pages/customer.py`)

- **Header:** "AI Support — Refund Assistant"
- **Chat Window:** Scrollable message history (user + agent)
- **Input:** Text box + microphone button (records audio, sends to `/voice` endpoint)
- **Status Indicators:** Typing… | Policy Check | CRM Lookup | Final Decision
- **Response Cards:** Green (approved), Red (denied), Yellow (escalated)

8.2 Admin Dashboard (`pages/admin.py`)

- **Real-Time Logs:** Auto-refreshing table of all agent runs (trace_id, customer, decision, latency)
- **Reasoning Trace Viewer:** Expandable JSON view of each node execution
- **Policy Violation Alerts:** Filtered view of all "DENY" decisions with cited policy clauses
- **Metrics Cards:** Total requests, approval rate, avg latency, escalation rate
- **CRM Lookup:** Search customer by ID to see full profile and history

8.3 Voice Component

- **Recording:** `st.audio_input()` (Streamlit native) or custom HTML/JS component
- **Playback:** Agent responses streamed via ElevenLabs with `st.audio()`
- **Fallback:** If voice fails, automatically fall back to text chat

---

9. Voice Pipeline (Bonus Feature)

Component	Tool	Flow
Speech-to-Text	OpenAI Whisper API	Audio blob → `/voice/transcribe` → text
LLM Processing	Groq LLaMA 3.3 70B	Same LangGraph loop as text
Text-to-Speech	ElevenLabs API v2	Agent response text → audio stream → frontend
Streaming	FastAPI StreamingResponse	Chunked audio delivery for low latency

Optimization: Cache common phrases ("Your refund is approved," "Policy requires…") to reduce TTS latency.

---

10. MLOps & Observability

10.1 MLflow Agent Tracing

- Log every LangGraph run as an MLflow experiment
- Track: `trace_id`, `node_execution_times`, `retrieved_documents`, `final_decision`, `confidence_score`
- Compare runs over time to detect drift in approval patterns

10.2 Prometheus Metrics

- `refund_requests_total` (counter, labels: decision_type)
- `agent_decision_latency_seconds` (histogram)
- `policy_retrieval_accuracy` (gauge, manual annotation)
- `escalation_rate` (gauge)
- `voice_pipeline_failures` (counter)

10.3 Grafana Dashboard

- **Panel 1:** Request volume over time (line graph)
- **Panel 2:** Decision distribution (pie chart: Approve/Deny/Escalate)
- **Panel 3:** P95 latency by node (heatmap)
- **Panel 4:** Active policy violations table (real-time)

---

11. CI/CD & Deployment

11.1 Docker Strategy

- **Multi-stage build:** Python 3.11 slim → dependencies → app layer
- **Size optimization:** Use `python:3.11-slim`, exclude dev dependencies, target < 1GB image
- **Services:** Single container with FastAPI + Streamlit (simpler for demo), or dual-container via Docker Compose

11.2 GitHub Actions Pipeline

```
# Conceptual workflow
on: push
jobs:
  test:
    - lint (ruff)
    - unit tests (pytest for tools, mock LLM calls)
  build:
    - docker build
    - push to GHCR
  deploy:
    - ssh / render deploy hook
```

11.3 Render Deployment

- **Web Service:** FastAPI backend (port 8000)
- **Static Site / Second Service:** Streamlit frontend (port 8501)
- **Environment Variables:** Groq API key, Pinecone API key, ElevenLabs key, etc.
- **Health Checks:** `/health` endpoint for Render uptime monitoring

---

12. 7-Day Execution Timeline

Day	Focus	Deliverable
Day 1	Project scaffolding, mock data creation, policy doc ingestion	SQLite CRM seeded, Pinecone index populated, repo structure ready
Day 2	Core LangGraph agent loop + tool registry	Working backend with 5 tools, manual API testing via curl
Day 3	RAG pipeline + policy retrieval tuning	Accurate policy clause retrieval for 10 test queries

Table 5 – continued

Day	Focus	Deliverable
Day 4	FastAPI endpoints + Streamlit customer chat	End-to-end text flow working, decision logic solid
Day 5	Admin dashboard + MLflow/Prometheus integration	Real-time logs visible, metrics emitting, Grafana dashboard live
Day 6	Voice pipeline integration + CI/CD + Docker	Voice demo functional, GitHub Actions green, Render deployed
Day 7	End-to-end testing, Loom video recording, README polish	3 test cases pass, video recorded, repo public and documented

---

13. Loom Video Script (7–10 Minutes)

13.1 Live Demo (4–5 min)

1. **Intro (30s):** "This is an AI Customer Support Agent for e-commerce refunds, built with LangGraph, FastAPI, and Streamlit."
2. **Standard Refund (90s):**
   - Show customer chat UI
   - Input: "I want to refund my headphones, order #12345"
   - Watch agent reason: CRM lookup → Policy check → Approve
   - Show admin dashboard reasoning log in real-time
3. **Policy Violation / "Hold the Line" (90s):**
   - Input: "I want to refund my swimsuit from 6 months ago"
   - Watch agent: CRM lookup → Policy retrieval (exclusion clause) → Deny with explanation
   - Show admin alert: "Policy violation — Final sale category"
4. **Voice Interaction (60s):** *(If implemented)*
   - Click microphone, speak refund request
   - Hear agent respond via voice
   - Show transcription accuracy in reasoning log

13.2 Code Tour (2–3 min)

1. **Repo Architecture:** Show `app/`, `agents/`, `tools/`, `frontend/`, `data/` directories
2. **LangGraph Loop:** Walk through `graph.py` — nodes, edges, conditional logic
3. **Tool Registry:** Highlight MCP-style tool definitions, decoupled integration
4. **Voice Handler:** Show `voice_pipeline.py` — Whisper → LLM → ElevenLabs flow
5. **Observability:** Show MLflow trace + Prometheus metrics endpoint + Grafana panel

13.3 Reasoning Logs & Failure Handling (1–2 min)

1. **Admin Dashboard:** Expand a trace, show each node output
2. **Failure Scenario:** Show a retry loop where RAG returned irrelevant context, agent self-corrected
3. **Metrics:** Show Grafana panel with latency spikes and how you identified the bottleneck

14. README Structure

```
# AI Customer Support Agent

## Overview
## Live Demo (Loom link)
## Architecture Diagram
## Tech Stack
## Quick Start (Docker)
## Project Structure
## Agent Design (LangGraph)
## Tool Reference
## Voice Pipeline
## Observability
## Testing
## Deployment
## License
```

---

15. Success Criteria & Edge Cases to Cover

Must-Demo in Video:

- Standard refund approved with CRM update
- Refund denied with specific policy clause citation
- Admin dashboard shows real-time reasoning trace
- Voice pipeline (if included) handles full spoken interaction
- Edge case escalated to human with summary

Technical Validation:

- Agent correctly retrieves policy via RAG (not just LLM memorization)
- Tool calling is dynamic (agent chooses which tools to call based on context)
- Reasoning logs are structured and human-readable
- System handles concurrent requests without state bleeding
- Docker container builds and runs in < 2 minutes

---

16. Risk Mitigation

Risk	Mitigation
Groq API rate limits	Implement request queue + fallback to Azure OpenAI
Pinecone free tier limits	Add ChromaDB local fallback; delete/recreate index if needed
Voice latency too high	Cache TTS for common phrases; fallback to text
Render free tier sleeps	Use Render paid tier for demo week ($7) or self-host on Azure VM

Table 6 – continued

Risk	Mitigation
LLM hallucinates policy	Strict RAG with `retrieval_required` flag; validate against source chunks

---

17. Final Checklist Before Submission

- GitHub repo is public with clean commit history
- README has setup instructions, architecture diagram, and Loom link
- `.env.example` provided (no real secrets)
- `docker-compose.yml` for one-command local launch
- All 15 CRM profiles tested with at least 3 scenarios each
- Loom video is 7–10 minutes, audio clear, code readable
- Admin dashboard accessible and auto-refreshes
- Voice pipeline tested with at least 3 utterances

This plan is designed to be executable in 7 days while showcasing your full stack of AI engineering skills: LangGraph agent design, RAG optimization, FastAPI microservices, Streamlit frontend development, voice AI integration, and production MLOps observability.